

## BIBLE MATPLOTLIB – RÉFÉRENCE EXHAUSTIVE DES FONCTIONS

LÉGENDE : [plt] = matplotlib.pyplot | [fig] = Figure | [ax] = Axes | [s] = Series

### IMPORTS ET CONFIGURATION DE BASE

DESCRIPTION : Imports standards et configuration initiale de matplotlib  
POURQUOI : Nécessaire avant toute utilisation de matplotlib.  
CONTEXTE : Début de tout script ou notebook utilisant matplotlib.

#### IMPORTS STANDARDS :

|                                                                            |                                     |
|----------------------------------------------------------------------------|-------------------------------------|
| import matplotlib.pyplot as plt                                            | Import principal (interface pyplot) |
| import matplotlib as mpl                                                   | Accès au module racine              |
| import matplotlib.figure as mfig                                           | Classe Figure directe               |
| import matplotlib.axes as maxes                                            | Classe Axes directe                 |
| import matplotlib.patches as mpatches                                      | Formes géométriques                 |
| import matplotlib.lines as mlines                                          | Lignes personnalisées               |
| import matplotlib.colors as mcolors                                        | Gestion des couleurs                |
| import matplotlib.cm as cm                                                 | Colormaps                           |
| import matplotlib.ticker as ticker                                         | Formatage des axes                  |
| import matplotlib.gridspec as gridspec                                     | Grilles avancées                    |
| import matplotlib.transforms as transforms                                 | Transformations de coordonnées      |
| import matplotlib.path as mpath                                            | Effets sur les tracés               |
| import matplotlib.animation as animation                                   | Animations                          |
| from matplotlib.patches import FancyArrowPatch, Rectangle, Circle, Ellipse |                                     |
| from matplotlib.collections import LineCollection, PathCollection          |                                     |
| from mpl_toolkits.axes_grid1 import make_axes_locatable                    | Colorbar avancée                    |
| from mpl_toolkits.mplot3d import Axes3D                                    | Graphiques 3D                       |

#### BACKENDS :

|                    |                                      |
|--------------------|--------------------------------------|
| mpl.use("Agg")     | Backend sans affichage (fichiers)    |
| mpl.use("TkAgg")   | Backend Tk (GUI)                     |
| mpl.use("Qt5Agg")  | Backend Qt5 (GUI)                    |
| mpl.get_backend()  | Affiche le backend actuel            |
| %matplotlib inline | Jupyter : affichage dans le notebook |
| %matplotlib widget | Jupyter : graphiques interactifs     |

#### STYLES GLOBAUX :

|                                          |                                    |
|------------------------------------------|------------------------------------|
| plt.style.use("seaborn-v0_8")            | Applique un style prédéfini        |
| plt.style.use("ggplot")                  | Style ggplot                       |
| plt.style.use("dark_background")         | Fond sombre                        |
| plt.style.use("bmh")                     | Style Bayesian Methods for Hackers |
| plt.style.use("fivethirtyeight")         | Style FiveThirtyEight              |
| plt.style.use("tableau-colorblind10")    | Style accessible daltoniens        |
| plt.style.available                      | Liste tous les styles disponibles  |
| with plt.style.context("seaborn-v0_8"):  | Style temporaire (context manager) |
| plt.plot(...)                            |                                    |
| mpl.rcParams["figure.figsize"] = (10, 6) | Taille par défaut des figures      |
| mpl.rcParams["font.size"] = 12           | Taille de police par défaut        |
| mpl.rcParams["font.family"] = "serif"    | Famille de police                  |
| mpl.rcParams["axes.grid"] = True         | Grille activée par défaut          |
| mpl.rcParams["axes.spines.top"] = False  | Supprime le bord supérieur         |
| mpl.rcParams["lines.linewidth"] = 1.5    | Épaisseur de ligne par défaut      |
| mpl.rcParams["savefig.dpi"] = 150        | DPI de sauvegarde par défaut       |
| mpl.rcParams["savefig.bbox"] = "tight"   | Recadrage auto à la sauvegarde     |
| mpl.rcParamsdefaults()                   | Remet les paramètres par défaut    |
| mpl.rc("axes", labelsiz=14, titlesiz=16) | Raccourci pour plusieurs rc        |

### [plt] Création de figures – figure, subplots, subplot\_mosaic

DESCRIPTION : Crée des figures et des axes (zones de tracé)  
POURQUOI : La Figure est le conteneur principal ; l'Axes est la zone de dessin.  
CONTEXTE : Début de tout graphique matplotlib.

#### — INTERFACE PYPLOT (RECOMMANDÉE POUR GRAPHIQUES SIMPLES) —

|                               |                                        |
|-------------------------------|----------------------------------------|
| plt.figure()                  | Crée une figure vide                   |
| plt.figure(figsize=(10, 6))   | Taille en pouces (largeur, hauteur)    |
| plt.figure(dpi=150)           | Résolution en points par pouce         |
| plt.figure(facecolor="white") | Couleur de fond                        |
| plt.figure(edgecolor="black") | Couleur du bord                        |
| plt.figure(tight_layout=True) | Ajustement automatique des marges      |
| plt.figure(num=1)             | Numéro de figure (réutilise si existe) |

|                                                                                                    |                                                  |
|----------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <code>plt.figure(num="mon_graphique")</code>                                                       | Nom de figure                                    |
| <code>plt.figure(constrained_layout=True)</code>                                                   | Layout automatique avancé                        |
| <code>plt.axes()</code>                                                                            | Crée un axes dans la figure courante             |
| <code>plt.axes([left,bottom,width,height])</code>                                                  | Axes à position/taille relatives [0-1]           |
| <code>plt.gca()</code>                                                                             | Retourne l'axes courant (Get Current Axes)       |
| <code>plt.gcf()</code>                                                                             | Retourne la figure courante (Get Current Fig)    |
| <code>plt.sca(ax)</code>                                                                           | Définit l'axes courant (Set Current Axes)        |
| <code>plt.scf(fig)</code>                                                                          | Définit la figure courante                       |
| <code>plt.subplot(nrows, ncols, index)</code>                                                      | Crée un axes dans une grille                     |
| <code>plt.subplot(2, 3, 1)</code>                                                                  | 2 lignes, 3 colonnes, 1er axes                   |
| <code>plt.subplot(231)</code>                                                                      | Notation compacte (2,3,1)                        |
| <code>plt.subplot(projection="polar")</code>                                                       | Axes polaire                                     |
| <code>plt.subplot(projection="3d")</code>                                                          | Axes 3D                                          |
| <code>plt.tight_layout()</code>                                                                    | Ajuste les marges pour éviter les superpositions |
| <code>plt.tight_layout(pad=1.5)</code>                                                             | Marge de rembourrage                             |
| <code>plt.tight_layout(h_pad=2.0)</code>                                                           | Rembourrage vertical                             |
| <code>plt.tight_layout(w_pad=2.0)</code>                                                           | Rembourrage horizontal                           |
| <code>plt.tight_layout(rect=[0,0,1,0.95])</code>                                                   | Limite la zone de layout                         |
| <code>plt.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.1, hspace=0.4, wspace=0.3)</code> | Ajustement manuel des marges                     |
| <code>plt.show()</code>                                                                            | Affiche la figure                                |
| <code>plt.close()</code>                                                                           | Ferme la figure courante                         |
| <code>plt.close("all")</code>                                                                      | Ferme toutes les figures                         |
| <code>plt.close(fig)</code>                                                                        | Ferme une figure spécifique                      |
| <code>plt.clf()</code>                                                                             | Efface la figure courante                        |
| <code>plt.cla()</code>                                                                             | Efface l'axes courant                            |
| <code>plt.ion()</code>                                                                             | Mode interactif activé                           |
| <code>plt.ioff()</code>                                                                            | Mode interactif désactivé                        |
| <code>plt.isinteractive()</code>                                                                   | Vrai si mode interactif                          |

#### SUBPLOTS (RECOMMANDÉ POUR PLUSIEURS AXES)

|                                                                    |                                         |
|--------------------------------------------------------------------|-----------------------------------------|
| <code>fig, ax = plt.subplots()</code>                              | 1 figure, 1 axes                        |
| <code>fig, axes = plt.subplots(2, 3)</code>                        | 2 lignes × 3 colonnes d'axes (array 2D) |
| <code>fig, axes = plt.subplots(1, 3)</code>                        | 1 ligne × 3 axes (array 1D)             |
| <code>fig, axes = plt.subplots(3, 1, sharex=True)</code>           | Axes avec axe X partagé                 |
| <code>fig, axes = plt.subplots(2, 2, sharey=True)</code>           | Axes avec axe Y partagé                 |
| <code>fig, axes = plt.subplots(2, 2, sharex="col")</code>          | Partage X par colonne                   |
| <code>fig, axes = plt.subplots(2, 2, sharey="row")</code>          | Partage Y par ligne                     |
| <code>fig, axes = plt.subplots(2, 3, figsize=(15,8))</code>        | Avec taille personnalisée               |
| <code>fig, axes = plt.subplots(2, 2, squeeze=False)</code>         | Toujours retourner tableau 2D           |
| <code>fig, axes = plt.subplots(2, 2, squeeze=True)</code>          | Supprime les dimensions inutiles        |
| <code>fig, axes = plt.subplots(1, 3, width_ratios=[1,2,1])</code>  | Ratios de largeur                       |
| <code>fig, axes = plt.subplots(3, 1, height_ratios=[2,1,1])</code> | Ratios de hauteur                       |
| <code>plt.subplots_adjust(hspace=0.4, wspace=0.3)</code>           | Espacement entre axes                   |

#### SUBPLOT\_MOSAIC (LAYOUTS COMPLEXES)

|                                                                     |                                      |
|---------------------------------------------------------------------|--------------------------------------|
| <code>fig, axd = plt.subplot_mosaic("AB;CC")</code>                 | Layout 2x2 : A, B en haut ; C en bas |
| <code>fig, axd = plt.subplot_mosaic([["A","B"], ["C","C"]])</code>  | Notation liste                       |
| <code>fig, axd = plt.subplot_mosaic("AB;.C")</code>                 | Point = espace vide                  |
| <code>fig, axd = plt.subplot_mosaic("AB;CC", figsize=(10,6))</code> |                                      |
| <code>axd["A"].plot(...)</code>                                     | Accès par clé                        |
| <code>axd["B"].plot(...)</code>                                     |                                      |

#### GRIDSPEC (CONTRÔLE PRÉCIS)

|                                                                                  |                                  |
|----------------------------------------------------------------------------------|----------------------------------|
| <code>gs = gridspec.GridSpec(2, 3)</code>                                        | Grille 2x3                       |
| <code>gs = gridspec.GridSpec(2, 3, width_ratios=[1,2,1])</code>                  | Avec ratios                      |
| <code>gs = gridspec.GridSpec(2, 3, hspace=0.4)</code>                            | Espacement vertical              |
| <code>ax1 = fig.add_subplot(gs[0, 0])</code>                                     | Axes en position (0,0)           |
| <code>ax2 = fig.add_subplot(gs[0, 1:])</code>                                    | Axes couvrant colonnes 1 et 2    |
| <code>ax3 = fig.add_subplot(gs[1, :])</code>                                     | Axes sur toute la deuxième ligne |
| <code>gs2 = gridspec.GridSpecFromSubplotSpec(2, 2, subplot_spec=gs[0, 1])</code> | GridSpec imbriqué                |

#### MÉTHODES DE LA FIGURE

|                                                        |                                  |
|--------------------------------------------------------|----------------------------------|
| <code>fig.add_subplot(2, 3, 1)</code>                  | Ajoute un axes à la figure       |
| <code>fig.add_axes([l, b, w, h])</code>                | Axes à position absolue relative |
| <code>fig.add_gridspec(2, 3)</code>                    | GridSpec depuis la figure        |
| <code>fig.set_size_inches(10, 6)</code>                | Redimensionne la figure          |
| <code>fig.set_dpi(150)</code>                          | Change le DPI                    |
| <code>fig.set_facecolor("white")</code>                | Couleur de fond                  |
| <code>fig.suptitle("Titre global", fontsize=16)</code> | Titre de la figure entière       |
| <code>fig.suptitle("Titre", y=1.02)</code>             | Position verticale du titre      |
| <code>fig.colorbar(im, ax=ax)</code>                   | Ajoute une colorbar à un axes    |

|                                                    |                                   |
|----------------------------------------------------|-----------------------------------|
| <code>fig.colorbar(im, ax=axes, shrink=0.6)</code> | Colorbar pour plusieurs axes      |
| <code>fig.legend(loc="upper right")</code>         | Légende globale de la figure      |
| <code>fig.tight_layout()</code>                    | Ajuste les marges                 |
| <code>fig.savefig("fig.png")</code>                | Sauvegarde la figure              |
| <code>fig.canvas.draw()</code>                     | Redessine la figure               |
| <code>fig.canvas.flush_events()</code>             | Vide les événements de la fenêtre |
| <code>fig.get_axes()</code>                        | Liste des axes de la figure       |

#### EXEMPLES :

```
fig, axes = plt.subplots(2, 3, figsize=(15, 8), sharex=True)
for i, ax in enumerate(axes.flat):
    ax.plot(x, y[i])
plt.tight_layout()
plt.savefig("résultat.png", dpi=150, bbox_inches="tight")
```

[ax] Graphiques de base – plot, scatter, bar, hist...

DESCRIPTION : Fonctions de tracé principales disponibles sur un objet Axes  
 POURQUOI : Visualiser les données sous différentes formes.  
 CONTEXTE : Analyse exploratoire, rapports, publications.

#### — LIGNES ET COURBES —

|                                                     |                                      |
|-----------------------------------------------------|--------------------------------------|
| <code>ax.plot(x, y)</code>                          | Courbe simple                        |
| <code>ax.plot(x, y, "r--")</code>                   | Couleur rouge, tirets                |
| <code>ax.plot(x, y, color="steelblue")</code>       | Couleur nommée                       |
| <code>ax.plot(x, y, color="#2ecc71")</code>         | Couleur hexadécimale                 |
| <code>ax.plot(x, y, linestyle="-")</code>           | Ligne continue                       |
| <code>ax.plot(x, y, linestyle="--")</code>          | Tirets                               |
| <code>ax.plot(x, y, linestyle="-.")</code>          | Tiret-point                          |
| <code>ax.plot(x, y, linestyle=":")</code>           | Pointillés                           |
| <code>ax.plot(x, y, linewidth=2.5)</code>           | Épaisseur de ligne                   |
| <code>ax.plot(x, y, lw=2.5)</code>                  | Alias de linewidth                   |
| <code>ax.plot(x, y, alpha=0.7)</code>               | Transparence (0=invisible, 1=opaque) |
| <code>ax.plot(x, y, marker="o")</code>              | Marqueur circulaire                  |
| <code>ax.plot(x, y, marker="s")</code>              | Marqueur carré                       |
| <code>ax.plot(x, y, marker="^")</code>              | Marqueur triangle                    |
| <code>ax.plot(x, y, marker="D")</code>              | Marqueur diamant                     |
| <code>ax.plot(x, y, marker="+")</code>              | Marqueur croix                       |
| <code>ax.plot(x, y, marker="x")</code>              | Marqueur x                           |
| <code>ax.plot(x, y, marker="*")</code>              | Marqueur étoile                      |
| <code>ax.plot(x, y, markersize=8)</code>            | Taille du marqueur                   |
| <code>ax.plot(x, y, ms=8)</code>                    | Alias de markersize                  |
| <code>ax.plot(x, y, markerfacecolor="red")</code>   | Couleur intérieure marqueur          |
| <code>ax.plot(x, y, markeredgecolor="black")</code> | Couleur bordure marqueur             |
| <code>ax.plot(x, y, markeredgewidth=1.5)</code>     | Épaisseur bordure marqueur           |
| <code>ax.plot(x, y, label="Série 1")</code>         | Étiquette pour la légende            |
| <code>ax.plot(x, y, zorder=3)</code>                | Ordre de superposition               |
| <code>ax.plot(x, y, drawstyle="steps-pre")</code>   | Style escalier (avant)               |
| <code>ax.plot(x, y, drawstyle="steps-post")</code>  | Style escalier (après)               |
| <code>ax.plot(x, y, drawstyle="steps-mid")</code>   | Style escalier (milieu)              |
| <code>ax.plot(x, y, solid_capstyle="round")</code>  | Extrémités arrondies                 |
| <code>ax.plot(x, y, dash_capstyle="butt")</code>    | Extrémités des tirets                |
| <code>ax.plot(x1, y1, x2, y2, x3, y3)</code>        | Plusieurs courbes en un appel        |
| <code>plt.plot(x, y)</code>                         | Alias via pyplot (axes courant)      |

#### — NUAGE DE POINTS —

|                                                          |                                         |
|----------------------------------------------------------|-----------------------------------------|
| <code>ax.scatter(x, y)</code>                            | Nuage de points simple                  |
| <code>ax.scatter(x, y, s=50)</code>                      | Taille des points (en pt <sup>2</sup> ) |
| <code>ax.scatter(x, y, s=tailles)</code>                 | Taille variable par point               |
| <code>ax.scatter(x, y, c="red")</code>                   | Couleur uniforme                        |
| <code>ax.scatter(x, y, c=valeurs)</code>                 | Couleur selon une variable              |
| <code>ax.scatter(x, y, c=valeurs, cmap="viridis")</code> | Colormap pour la couleur                |
| <code>ax.scatter(x, y, c=valeurs, vmin=0, vmax=1)</code> | Plage de couleur                        |
| <code>ax.scatter(x, y, alpha=0.5)</code>                 | Transparence                            |
| <code>ax.scatter(x, y, edgecolors="black")</code>        | Bordure des points                      |
| <code>ax.scatter(x, y, linewidths=0.5)</code>            | Épaisseur de bordure                    |
| <code>ax.scatter(x, y, marker="^")</code>                | Marqueur personnalisé                   |
| <code>ax.scatter(x, y, label="Points")</code>            | Étiquette légende                       |
| <code>sc = ax.scatter(x, y, c=z)</code>                  | Retourne le PathCollection              |
| <code>plt.colorbar(sc, ax=ax)</code>                     | Colorbar depuis le scatter              |

#### — GRAPHIQUES EN BARRES —

|                                           |                    |
|-------------------------------------------|--------------------|
| <code>ax.bar(x, height)</code>            | Barres verticales  |
| <code>ax.bar(x, height, width=0.8)</code> | Largeur des barres |

|                                                       |                                 |
|-------------------------------------------------------|---------------------------------|
| <code>ax.bar(x, height, color="steelblue")</code>     | Couleur                         |
| <code>ax.bar(x, height, color=["r","g","b"])</code>   | Couleur différente par barre    |
| <code>ax.bar(x, height, edgecolor="black")</code>     | Bordure des barres              |
| <code>ax.bar(x, height, linewidth=0.5)</code>         | Épaisseur de bordure            |
| <code>ax.bar(x, height, alpha=0.7)</code>             | Transparence                    |
| <code>ax.bar(x, height, bottom=base)</code>           | Barres empilées (base)          |
| <code>ax.bar(x, height, align="center")</code>        | Alignement (défaut)             |
| <code>ax.bar(x, height, align="edge")</code>          | Alignement sur le bord gauche   |
| <code>ax.bar(x, height, capsize=5)</code>             | Taille des barres d'erreur      |
| <code>ax.bar(x, height, yerr=erreurs)</code>          | Barres d'erreur verticales      |
| <code>ax.bar(x, height, xerr=erreurs)</code>          | Barres d'erreur horizontales    |
| <code>ax.bar(x, height, log=True)</code>              | Échelle logarithmique           |
| <code>ax.bar(x, height, tick_label=étiquettes)</code> | Étiquettes des ticks            |
| <code>ax.barh(y, width)</code>                        | Barres horizontales             |
| <code>ax.barh(y, width, height=0.8)</code>            | Hauteur des barres horizontales |
| <code>ax.barh(y, width, left=base)</code>             | Barres empilées horizontales    |
| <code>ax.barh(y, width, xerr=erreurs)</code>          | Barres d'erreur horizontales    |

---

#### — HISTOGRAMMES —

|                                                      |                               |
|------------------------------------------------------|-------------------------------|
| <code>ax.hist(data)</code>                           | Histogramme simple            |
| <code>ax.hist(data, bins=20)</code>                  | Nombre de classes             |
| <code>ax.hist(data, bins="auto")</code>              | Nombre de classes automatique |
| <code>ax.hist(data, bins="sturges")</code>           | Règle de Sturges              |
| <code>ax.hist(data, bins="fd")</code>                | Règle de Freedman-Diaconis    |
| <code>ax.hist(data, bins=[0,10,20,50,100])</code>    | Classes personnalisées        |
| <code>ax.hist(data, density=True)</code>             | Densité (somme = 1)           |
| <code>ax.hist(data, cumulative=True)</code>          | Histogramme cumulé            |
| <code>ax.hist(data, histtype="bar")</code>           | Type barre (défaut)           |
| <code>ax.hist(data, histtype="step")</code>          | Contour seulement             |
| <code>ax.hist(data, histtype="stepfilled")</code>    | Contour rempli                |
| <code>ax.hist(data, histtype="barstacked")</code>    | Barres empilées               |
| <code>ax.hist(data, color="steelblue")</code>        | Couleur                       |
| <code>ax.hist(data, edgecolor="black")</code>        | Bordure des barres            |
| <code>ax.hist(data, alpha=0.7)</code>                | Transparence                  |
| <code>ax.hist(data, orientation="horizontal")</code> | Histogramme horizontal        |
| <code>ax.hist(data, log=True)</code>                 | Axe Y logarithmique           |
| <code>ax.hist(data, range=(0, 100))</code>           | Plage des données             |
| <code>ax.hist(data, weights=poids)</code>            | Histogramme pondéré           |
| <code>ax.hist(data, rwidth=0.8)</code>               | Largeur relative des barres   |
| <code>ax.hist([d1, d2, d3], bins=20)</code>          | Plusieurs datasets superposés |
| <code>ax.hist([d1, d2], stacked=True)</code>         | Histogrammes empilés          |
| <code>ax.hist(data, label="Distribution")</code>     | Étiquette légende             |
| <code>n, bins, patches = ax.hist(data)</code>        | Récupère les valeurs          |

---

#### — GRAPHIQUES EN BOÎTES —

|                                                                |                                     |
|----------------------------------------------------------------|-------------------------------------|
| <code>ax.boxplot(data)</code>                                  | Boîte à moustaches                  |
| <code>ax.boxplot([d1, d2, d3])</code>                          | Plusieurs boîtes                    |
| <code>ax.boxplot(data, vert=False)</code>                      | Boîtes horizontales                 |
| <code>ax.boxplot(data, notch=True)</code>                      | Avec encoche (IC médiane)           |
| <code>ax.boxplot(data, patch_artist=True)</code>               | Boîtes remplies (pas juste contour) |
| <code>ax.boxplot(data, showfliers=True)</code>                 | Affiche les valeurs aberrantes      |
| <code>ax.boxplot(data, showfliers=False)</code>                | Masque les valeurs aberrantes       |
| <code>ax.boxplot(data, showmeans=True)</code>                  | Affiche la moyenne                  |
| <code>ax.boxplot(data, meanline=True)</code>                   | Moyenne comme ligne                 |
| <code>ax.boxplot(data, whis=1.5)</code>                        | Facteur des moustaches (défaut 1.5) |
| <code>ax.boxplot(data, whis=(5, 95))</code>                    | Moustaches aux percentiles          |
| <code>ax.boxplot(data, widths=0.7)</code>                      | Largeur des boîtes                  |
| <code>ax.boxplot(data, labels=["A","B","C"])</code>            | Étiquettes des boîtes               |
| <code>ax.boxplot(data, positions=[1,3,5])</code>               | Positions des boîtes                |
| <code>ax.boxplot(data, sym="r+")</code>                        | Style des valeurs aberrantes        |
| <code>ax.boxplot(data, medianprops={"color":"red"})</code>     | Style de la médiane                 |
| <code>ax.boxplot(data, boxprops={"color":"blue"})</code>       | Style des boîtes                    |
| <code>ax.boxplot(data, whiskerprops={"linestyle":"--"})</code> | Style des moustaches                |
| <code>ax.boxplot(data, flierprops={"marker":"o"})</code>       | Style des aberrants                 |
| <code>ax.violinplot(data)</code>                               | Diagramme en violon                 |
| <code>ax.violinplot(data, showmedians=True)</code>             | Avec médiane                        |
| <code>ax.violinplot(data, showmeans=True)</code>               | Avec moyenne                        |
| <code>ax.violinplot(data, showextrema=True)</code>             | Avec extrêmes                       |
| <code>ax.violinplot(data, vert=False)</code>                   | Violons horizontaux                 |
| <code>ax.violinplot(data, positions=[1,2,3])</code>            | Positions                           |

---

#### — GRAPHIQUES D'ERREUR —

|                                                     |                              |
|-----------------------------------------------------|------------------------------|
| <code>ax.errorbar(x, y, yerr=err)</code>            | Barres d'erreur verticales   |
| <code>ax.errorbar(x, y, xerr=err)</code>            | Barres d'erreur horizontales |
| <code>ax.errorbar(x, y, yerr=[lower, upper])</code> | Barres asymétriques          |

|                                                          |                               |
|----------------------------------------------------------|-------------------------------|
| <code>ax.errorbar(x, y, yerr=err, fmt="o-")</code>       | Avec format de courbe         |
| <code>ax.errorbar(x, y, yerr=err, capsize=5)</code>      | Taille des embouts            |
| <code>ax.errorbar(x, y, yerr=err, capthick=2)</code>     | Épaisseur des embouts         |
| <code>ax.errorbar(x, y, yerr=err, ecolor="red")</code>   | Couleur des barres d'erreur   |
| <code>ax.errorbar(x, y, yerr=err, elinewidth=1.5)</code> | Épaisseur des barres d'erreur |
| <code>ax.errorbar(x, y, yerr=err, barsabove=True)</code> | Barres au-dessus des symboles |

---

#### — ZONES REMPLIES —

|                                                           |                                    |
|-----------------------------------------------------------|------------------------------------|
| <code>ax.fill_between(x, y1, y2)</code>                   | Remplit entre deux courbes         |
| <code>ax.fill_between(x, y1, y2, color="blue")</code>     | Avec couleur                       |
| <code>ax.fill_between(x, y1, y2, alpha=0.3)</code>        | Avec transparence                  |
| <code>ax.fill_between(x, y1, y2, where=condition)</code>  | Remplissage conditionnel           |
| <code>ax.fill_between(x, y1, y2, interpolate=True)</code> | Interpolation aux intersections    |
| <code>ax.fill_between(x, y1, y2, label="IC 95%")</code>   | Étiquette légende                  |
| <code>ax.fill_betweenx(y, x1, x2)</code>                  | Remplit entre deux courbes (axe Y) |
| <code>ax.fill(x, y, color="green", alpha=0.5)</code>      | Remplit un polygone fermé          |

---

#### — GRAPHIQUES EN CAMEMBERT —

|                                                            |                                     |
|------------------------------------------------------------|-------------------------------------|
| <code>ax.pie(valeurs)</code>                               | Camembert simple                    |
| <code>ax.pie(valeurs, labels=étiquettes)</code>            | Avec étiquettes                     |
| <code>ax.pie(valeurs, autopct="%1.1f%")</code>             | Pourcentages sur les parts          |
| <code>ax.pie(valeurs, autopct="%1.0f%")</code>             | Pourcentages sans décimale          |
| <code>ax.pie(valeurs, explode=[0.1,0,0,0])</code>          | Détache une part                    |
| <code>ax.pie(valeurs, colors=["r","g","b","y"])</code>     | Couleurs personnalisées             |
| <code>ax.pie(valeurs, shadow=True)</code>                  | Ombre                               |
| <code>ax.pie(valeurs, startangle=90)</code>                | Angle de départ                     |
| <code>ax.pie(valeurs, counterclock=False)</code>           | Sens horaire                        |
| <code>ax.pie(valeurs, wedgeprops={"edgecolor":"w"})</code> | Style des parts                     |
| <code>ax.pie(valeurs, textprops={"fontsize":12})</code>    | Style du texte                      |
| <code>ax.pie(valeurs, radius=1.2)</code>                   | Rayon du camembert                  |
| <code>ax.pie(valeurs, normalize=True)</code>               | Normalise les valeurs               |
| <code>ax.pie(valeurs, pctdistance=0.85)</code>             | Distance des pourcentages au centre |
| <code>ax.pie(valeurs, labeldistance=1.1)</code>            | Distance des étiquettes au centre   |

---

#### — GRAPHIQUES SPÉCIAUX —

|                                                                    |                                 |
|--------------------------------------------------------------------|---------------------------------|
| <code>ax.stem(x, y)</code>                                         | Graphique en bâtons             |
| <code>ax.stem(x, y, linefmt="b-")</code>                           | Format des tiges                |
| <code>ax.stem(x, y, markerfmt="bo")</code>                         | Format des marqueurs            |
| <code>ax.stem(x, y, basefmt="r-")</code>                           | Format de la ligne de base      |
| <code>ax.step(x, y)</code>                                         | Graphique en escalier           |
| <code>ax.step(x, y, where="pre")</code>                            | Marche avant le point           |
| <code>ax.step(x, y, where="post")</code>                           | Marche après le point           |
| <code>ax.step(x, y, where="mid")</code>                            | Marche au milieu                |
| <code>ax.eventplot(positions)</code>                               | Événements discrets             |
| <code>ax.eventplot(positions, orientation="vertical")</code>       | Vertical                        |
| <code>ax.eventplot(positions, colors=couleurs)</code>              | Couleurs par événement          |
| <code>ax.broken_barh([(0,5),(10,3)], (5,2))</code>                 | Barres discontinues             |
| <code>ax.hexbin(x, y)</code>                                       | Graphique hexagonal             |
| <code>ax.hexbin(x, y, gridsize=30)</code>                          | Taille de la grille             |
| <code>ax.hexbin(x, y, cmap="Blues")</code>                         | Colormap                        |
| <code>ax.hexbin(x, y, C=valeurs, reduce_C_function=np.mean)</code> | Valeurs agrégées                |
| <code>ax.hexbin(x, y, bins="log")</code>                           | Échelle log                     |
| <code>ax.quiver(X, Y, U, V)</code>                                 | Champ de vecteurs               |
| <code>ax.quiver(X, Y, U, V, scale=1)</code>                        | Échelle des vecteurs            |
| <code>ax.quiver(X, Y, U, V, color="red")</code>                    | Couleur                         |
| <code>ax.streamplot(X, Y, U, V)</code>                             | Lignes de flux                  |
| <code>ax.streamplot(X, Y, U, V, density=1.5)</code>                | Densité des lignes              |
| <code>ax.streamplot(X, Y, U, V, color=V)</code>                    | Couleur selon vitesse           |
| <code>ax.contour(X, Y, Z)</code>                                   | Courbes de niveau               |
| <code>ax.contour(X, Y, Z, levels=10)</code>                        | Nombre de niveaux               |
| <code>ax.contour(X, Y, Z, levels=[0,1,2,5])</code>                 | Niveaux explicites              |
| <code>ax.contour(X, Y, Z, cmap="viridis")</code>                   | Colormap                        |
| <code>ax.contourf(X, Y, Z)</code>                                  | Courbes de niveau remplies      |
| <code>ax.contourf(X, Y, Z, levels=20)</code>                       | Avec remplissage                |
| <code>ax.imshow(image)</code>                                      | Affiche une image/matrice       |
| <code>ax.imshow(image, cmap="gray")</code>                         | En niveaux de gris              |
| <code>ax.imshow(image, interpolation="nearest")</code>             | Sans interpolation              |
| <code>ax.imshow(image, aspect="equal")</code>                      | Rapport d'aspect                |
| <code>ax.imshow(image, origin="upper")</code>                      | Origine en haut (défaut images) |
| <code>ax.imshow(image, origin="lower")</code>                      | Origine en bas (maths)          |
| <code>ax.imshow(image, extent=[x0,x1,y0,y1])</code>                | Étendue des axes                |
| <code>ax.imshow(image, vmin=0, vmax=1)</code>                      | Plage de couleur                |
| <code>ax.pcolor(X, Y, Z)</code>                                    | Pseudo-couleur                  |
| <code>ax.pcolormesh(X, Y, Z)</code>                                | Pseudo-couleur optimisée        |
| <code>ax.pcolormesh(X, Y, Z, cmap="RdBu")</code>                   | Colormap divergente             |

|                                                     |                                       |
|-----------------------------------------------------|---------------------------------------|
| <code>ax.pcolormesh(X, Y, Z, shading="auto")</code> | Shading automatique                   |
| <code>ax.matshow(matrice)</code>                    | Affiche une matrice numérique         |
| <code>ax.spy(matrice_creuse)</code>                 | Matrice creuse (positions non-nulles) |
| <code>ax.barbs(X, Y, U, V)</code>                   | Girouettes météo                      |
| <code>ax.cohere(x, y)</code>                        | Cohérence entre deux signaux          |
| <code>ax.csd(x, y)</code>                           | Densité spectrale croisée             |
| <code>ax.psd(x)</code>                              | Densité spectrale de puissance        |
| <code>ax.specgram(x, Fs=1000)</code>                | Spectrogramme                         |
| <code>ax.acorr(x)</code>                            | Autocorrélation                       |
| <code>ax.xcorr(x, y)</code>                         | Corrélation croisée                   |
| <code>ax.magnitude_spectrum(x)</code>               | Spectre de magnitude                  |

```

EXEMPLES      :
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot(x, y, "b-o", lw=2, ms=8, label="Mesures")
ax.fill_between(x, y-err, y+err, alpha=0.2, label="IC 95%")
ax.legend()
plt.tight_layout()

```

[ax] Axes, titres, étiquettes – `set_*`, `xlabel`, `ylabel`, `title`

DESCRIPTION : Configure les axes, ajoute des titres et étiquettes  
POURQUOI : Rendre les graphiques lisibles et informatifs.  
CONTEXTE : Finition de tout graphique pour présentation ou publication.

#### — TITRES ET ÉTIQUETTES —

|                                                                                      |                                 |
|--------------------------------------------------------------------------------------|---------------------------------|
| <code>ax.set_title("Titre")</code>                                                   | Titre du graphique              |
| <code>ax.set_title("Titre", fontsize=16)</code>                                      | Taille de police                |
| <code>ax.set_title("Titre", fontweight="bold")</code>                                | Gras                            |
| <code>ax.set_title("Titre", fontstyle="italic")</code>                               | Italique                        |
| <code>ax.set_title("Titre", color="navy")</code>                                     | Couleur                         |
| <code>ax.set_title("Titre", pad=20)</code>                                           | Espace entre titre et graphique |
| <code>ax.set_title("Titre", loc="left")</code>                                       | Position : left, center, right  |
| <code>ax.set_title("Titre \$mu\$ = 5")</code>                                        | Avec formule LaTeX              |
| <code>ax.set_xlabel("Axe X")</code>                                                  | Étiquette axe X                 |
| <code>ax.set_xlabel("Axe X", fontsize=14)</code>                                     | Avec taille de police           |
| <code>ax.set_xlabel("Axe X", labelpad=10)</code>                                     | Espace étiquette ↔ axe          |
| <code>ax.set_ylabel("Axe Y")</code>                                                  | Étiquette axe Y                 |
| <code>ax.set_ylabel("Axe Y", rotation=0)</code>                                      | Étiquette horizontale           |
| <code>ax.set_ylabel("Axe Y", labelpad=15)</code>                                     | Espace étiquette ↔ axe          |
| # Méthode raccourci pour tout définir en une fois                                    |                                 |
| <code>ax.set(title="Titre", xlabel="X", ylabel="Y", xlim=(0,10), ylim=(-1,1))</code> |                                 |

#### — LIMITES DES AXES —

|                                               |                                          |
|-----------------------------------------------|------------------------------------------|
| <code>ax.set_xlim(0, 10)</code>               | Limites de l'axe X                       |
| <code>ax.set_xlim(left=0)</code>              | Seulement la borne gauche                |
| <code>ax.set_xlim(right=10)</code>            | Seulement la borne droite                |
| <code>ax.set_ylim(-1, 1)</code>               | Limites de l'axe Y                       |
| <code>ax.set_ylim(bottom=0)</code>            | Seulement la borne inférieure            |
| <code>ax.get_xlim()</code>                    | Retourne les limites X actuelles         |
| <code>ax.get_ylim()</code>                    | Retourne les limites Y actuelles         |
| <code>ax.autoscale()</code>                   | Ajuste automatiquement les limites       |
| <code>ax.autoscale(axis="x")</code>           | Seulement l'axe X                        |
| <code>ax.autoscale(tight=True)</code>         | Ajustement au plus proche                |
| <code>ax.margins(0.1)</code>                  | Marges proportionnelles                  |
| <code>ax.margins(x=0.05, y=0.1)</code>        | Marges différentes par axe               |
| <code>ax.relim()</code>                       | Recalcule les limites depuis les données |
| <code>ax.set_xbound(lower=0, upper=10)</code> | Borne sans modifier l'autoscale          |
| <code>ax.invert_xaxis()</code>                | Inverse l'axe X                          |
| <code>ax.invert_yaxis()</code>                | Inverse l'axe Y                          |
| <code>ax.xaxis_inverted()</code>              | Vrai si l'axe X est inversé              |

#### — ÉCHELLES —

|                                                         |                               |
|---------------------------------------------------------|-------------------------------|
| <code>ax.set_xscale("linear")</code>                    | Échelle linéaire (défaut)     |
| <code>ax.set_xscale("log")</code>                       | Échelle logarithmique         |
| <code>ax.set_xscale("log", base=2)</code>               | Log base 2                    |
| <code>ax.set_xscale("symlog")</code>                    | Log symétrique (valeurs neg.) |
| <code>ax.set_xscale("symlog", linthresh=0.01)</code>    | Seuil linéaire pour symlog    |
| <code>ax.set_xscale("logit")</code>                     | Échelle logit                 |
| <code>ax.set_xscale("function", functions=(f,g))</code> | Échelle personnalisée         |
| <code>ax.set_yscale("log")</code>                       | Axe Y logarithmique           |
| <code>ax.set_aspect("equal")</code>                     | Rapport d'aspect 1:1          |
| <code>ax.set_aspect("auto")</code>                      | Rapport d'aspect automatique  |
| <code>ax.set_aspect(2.0)</code>                         | Rapport d'aspect numérique    |

|                                              |                    |
|----------------------------------------------|--------------------|
| ax.set_aspect("equal", adjustable="box")     | Ajuste la boîte    |
| ax.set_aspect("equal", adjustable="datalim") | Ajuste les données |

#### — TICKS (GRADUATIONS) —

|                                                                             |                                 |
|-----------------------------------------------------------------------------|---------------------------------|
| ax.set_xticks([0, 1, 2, 3])                                                 | Positions des ticks X           |
| ax.set_xticks(np.arange(0, 10, 0.5))                                        | Ticks réguliers                 |
| ax.set_xticklabels(["A","B","C"])                                           | Étiquettes des ticks X          |
| ax.set_xticklabels(étiquettes, rotation=45)                                 | Étiquettes tournées             |
| ax.set_xticklabels(étiquettes, ha="right")                                  | Alignement horizontal           |
| ax.set_xticklabels(étiquettes, fontsize=10)                                 | Taille de police                |
| ax.set_yticks([0, 0.5, 1.0])                                                | Positions des ticks Y           |
| ax.set_yticklabels([])                                                      | Supprime les étiquettes Y       |
| ax.tick_params(axis="x", rotation=45)                                       | Rotation des ticks              |
| ax.tick_params(axis="both", labelsz=10)                                     | Taille des étiquettes           |
| ax.tick_params(axis="x", direction="in")                                    | Ticks vers l'intérieur          |
| ax.tick_params(axis="x", direction="out")                                   | Ticks vers l'extérieur (défaut) |
| ax.tick_params(axis="x", length=5)                                          | Longueur des ticks              |
| ax.tick_params(axis="x", width=1.5)                                         | Épaisseur des ticks             |
| ax.tick_params(axis="x", color="red")                                       | Couleur des ticks               |
| ax.tick_params(axis="x", pad=8)                                             | Espace ticks ↔ étiquettes       |
| ax.tick_params(top=True, right=True)                                        | Ticks sur les 4 côtés           |
| ax.tick_params(which="minor", length=3)                                     | Style des ticks mineurs         |
| ax.minorticks_on()                                                          | Active les ticks mineurs        |
| ax.minorticks_off()                                                         | Désactive les ticks mineurs     |
| # Via le module ticker (contrôle avancé)                                    |                                 |
| ax.xaxis.set_major_locator(ticker.MultipleLocator(5))                       | Ticks tous les 5                |
| ax.xaxis.set_major_locator(ticker.MaxNLocator(10))                          | Max 10 ticks                    |
| ax.xaxis.set_major_locator(ticker.LogLocator())                             | Ticks log                       |
| ax.xaxis.set_major_locator(ticker.AutoLocator())                            | Automatique                     |
| ax.xaxis.set_major_locator(ticker.FixedLocator([1,3,5]))                    | Positions fixes                 |
| ax.xaxis.set_major_locator(ticker.NullLocator())                            | Aucun tick                      |
| ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())                       | Ticks mineurs auto              |
| ax.xaxis.set_minor_locator(ticker.MultipleLocator(1))                       | Ticks mineurs tous les 1        |
| ax.xaxis.set_major_formatter(ticker.FormatStrFormatter("%.2f"))             | Format printf                   |
| ax.xaxis.set_major_formatter(ticker.FuncFormatter(lambda x,p: f"{x:.1f}€")) | Format personnalisé             |
| ax.xaxis.set_major_formatter(ticker.PercentFormatter(xmax=1))               | Pourcentages                    |
| ax.xaxis.set_major_formatter(ticker.ScalarFormatter())                      | Scalaire standard               |
| ax.xaxis.set_major_formatter(ticker.NullFormatter())                        | Supprime étiquettes             |
| ax.xaxis.set_major_formatter(mpl.dates.DateFormatter("%d/%m"))              | Dates                           |
| plt.xticks(rotation=45, ha="right")                                         | Via pyplot (axes courant)       |
| plt.yticks(fontsize=10)                                                     | Via pyplot                      |

#### — GRILLE —

|                         |                                     |
|-------------------------|-------------------------------------|
| ax.grid(True)           | Active la grille                    |
| ax.grid(False)          | Désactive la grille                 |
| ax.grid(axis="x")       | Grille verticale seulement          |
| ax.grid(axis="y")       | Grille horizontale seulement        |
| ax.grid(color="gray")   | Couleur de la grille                |
| ax.grid(linestyle="--") | Style de ligne                      |
| ax.grid(linewidth=0.5)  | Épaisseur de ligne                  |
| ax.grid(alpha=0.5)      | Transparence                        |
| ax.grid(which="major")  | Grille sur les ticks majeurs        |
| ax.grid(which="minor")  | Grille sur les ticks mineurs        |
| ax.grid(which="both")   | Grille sur tous les ticks           |
| ax.grid(zorder=0)       | Ordre (mettre derrière les données) |

#### — SPINES (BORDURES DES AXES) —

|                                                |                                  |
|------------------------------------------------|----------------------------------|
| ax.spines["top"].set_visible(False)            | Supprime le bord supérieur       |
| ax.spines["right"].set_visible(False)          | Supprime le bord droit           |
| ax.spines["bottom"].set_visible(True)          | Affiche le bord inférieur        |
| ax.spines["left"].set_color("red")             | Couleur du bord gauche           |
| ax.spines["bottom"].set_linewidth(2)           | Épaisseur du bord                |
| ax.spines["left"].set_linestyle("--")          | Style de ligne                   |
| ax.spines["left"].set_position(("outward",10)) | Déplace le bord vers l'extérieur |
| ax.spines["bottom"].set_bounds(0, 10)          | Borne le bord aux données        |
| for spine in ax.spines.values():               | Tous les bords                   |
| spine.set_visible(False)                       |                                  |

#### — EXEMPLES —

```
ax.set_title("Évolution mensuelle", fontsize=16, fontweight="bold", pad=15)
ax.set_xlabel("Mois", fontsize=13)
ax.set_ylabel("Ventes (k€)", fontsize=13)
ax.set_xlim(0, 12)
ax.set_ylim(bottom=0)
ax.tick_params(axis="x", rotation=45)
```

```
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)
ax.grid(axis="y", alpha=0.4)
```

[ax] Légende – legend, get\_legend\_handles\_labels

DESCRIPTION : Ajoute et configure la légende d'un graphique  
 POURQUOI : Identifier les différentes séries ou éléments du graphique.  
 CONTEXTE : Graphiques multi-séries, comparaisons.

#### FONCTIONS PRINCIPALES :

|                                                                                     |                                     |
|-------------------------------------------------------------------------------------|-------------------------------------|
| ax.legend()                                                                         | Légende automatique (depuis labels) |
| ax.legend(loc="upper right")                                                        | Position prédéfinie                 |
| ax.legend(loc="upper left")                                                         | En haut à gauche                    |
| ax.legend(loc="lower right")                                                        | En bas à droite                     |
| ax.legend(loc="center")                                                             | Centré                              |
| ax.legend(loc="best")                                                               | Meilleure position auto             |
| ax.legend(loc=(0.7, 0.2))                                                           | Position manuelle (x, y) relatives  |
| ax.legend(bbox_to_anchor=(1, 1))                                                    | Ancrage hors du graphique           |
| ax.legend(bbox_to_anchor=(0.5, -0.1),<br>loc="upper center")                        | En dessous, centré                  |
| ax.legend(ncols=2)                                                                  | Légende sur 2 colonnes              |
| ax.legend(ncols=3, columnspacing=1.0)                                               | Espacement entre colonnes           |
| ax.legend(fontsize=12)                                                              | Taille de police                    |
| ax.legend(title="Séries")                                                           | Titre de la légende                 |
| ax.legend(title="Séries", title_fontsize=13)                                        | Avec taille du titre                |
| ax.legend(frameon=True)                                                             | Avec cadre (défaut)                 |
| ax.legend(frameon=False)                                                            | Sans cadre                          |
| ax.legend(framealpha=0.8)                                                           | Transparence du cadre               |
| ax.legend(facecolor="white")                                                        | Couleur de fond                     |
| ax.legend(edgecolor="gray")                                                         | Couleur du bord du cadre            |
| ax.legend(shadow=True)                                                              | Ombre                               |
| ax.legend(fancybox=True)                                                            | Bords arrondis                      |
| ax.legend(markerscale=1.5)                                                          | Taille des symboles dans la légende |
| ax.legend(handlelength=2.0)                                                         | Longueur des lignes de symbole      |
| ax.legend(handleheight=1.0)                                                         | Hauteur des symboles                |
| ax.legend(handletextpad=0.5)                                                        | Espace symbole ↔ texte              |
| ax.legend(labelspacing=0.5)                                                         | Espace entre les entrées            |
| ax.legend(borderpad=1.0)                                                            | Rembourrage intérieur               |
| ax.legend(reverse=True)                                                             | Inverser l'ordre des entrées        |
| ax.legend(handles=[line1,line2],<br>labels=["A","B"])                               | Handles et labels manuels           |
| ax.legend(handles, labels)                                                          | Depuis get_legend_handles_labels    |
| handles, labels = ax.get_legend_handles_labels()                                    | Récupère les éléments               |
| leg = ax.legend()                                                                   | Retourne l'objet Legend             |
| leg.get_frame().set_alpha(0.5)                                                      | Modifie le cadre a posteriori       |
| leg.set_title("Titre")                                                              | Modifie le titre a posteriori       |
| plt.legend()                                                                        | Via pyplot (axes courant)           |
| # Légende avec éléments personnalisés                                               |                                     |
| from matplotlib.lines import Line2D                                                 |                                     |
| custom = [Line2D([0],[0], color="r", lw=2, label="Rouge")]                          |                                     |
| ax.legend(handles=custom)                                                           |                                     |
| # Légende multi-axes (sur la figure)                                                |                                     |
| fig.legend(loc="upper right", bbox_to_anchor=(1,1),<br>bbox_transform=ax.transAxes) |                                     |

EXEMPLES :

```
ax.legend(loc="upper left", fontsize=11, framealpha=0.9,  
ncols=2, title="Régions")
```

[ax] Texte et annotations – text, annotate, axvline, axhline

DESCRIPTION : Ajoute du texte, des flèches et des lignes de référence  
 POURQUOI : Mettre en valeur des points, expliquer des valeurs, ajouter du contexte.  
 CONTEXTE : Mise en forme finale, présentations, rapports.

#### — TEXTE —

|                                            |                            |
|--------------------------------------------|----------------------------|
| ax.text(x, y, "Texte")                     | Texte à la position (x, y) |
| ax.text(x, y, "Texte", fontsize=12)        | Taille de police           |
| ax.text(x, y, "Texte", color="red")        | Couleur                    |
| ax.text(x, y, "Texte", fontweight="bold")  | Gras                       |
| ax.text(x, y, "Texte", fontstyle="italic") | Italique                   |
| ax.text(x, y, "Texte", ha="center")        | Alignement horizontal      |
| ax.text(x, y, "Texte", ha="left")          | Alignement gauche          |



|                                                                                                    |                                |
|----------------------------------------------------------------------------------------------------|--------------------------------|
| <code>ax.text(x, y, "Texte", ha="right")</code>                                                    | Alignement droit               |
| <code>ax.text(x, y, "Texte", va="center")</code>                                                   | Alignement vertical            |
| <code>ax.text(x, y, "Texte", va="top")</code>                                                      | En haut                        |
| <code>ax.text(x, y, "Texte", va="bottom")</code>                                                   | En bas                         |
| <code>ax.text(x, y, "Texte", rotation=45)</code>                                                   | Rotation du texte              |
| <code>ax.text(x, y, "Texte", alpha=0.5)</code>                                                     | Transparence                   |
| <code>ax.text(x, y, "Texte", zorder=5)</code>                                                      | Ordre de superposition         |
| <code>ax.text(x, y, r"\$\mu = \bar{x}\$")</code>                                                   | Formule LaTeX                  |
| <code>ax.text(x, y, "Texte",<br/>bbox=dict(boxstyle="round", facecolor="wheat", alpha=0.5))</code> | Avec boîte                     |
| <code>ax.text(x, y, "Texte",<br/>transform=ax.transAxes)</code>                                    | Coordonnées relatives (0-1)    |
| <code>ax.text(0.5, 0.95, "Titre",<br/>transform=ax.transAxes, ha="center", va="top")</code>        | Texte en coordonnées d'axes    |
| <code>fig.text(x, y, "Texte figure")</code>                                                        | Texte en coordonnées de figure |

#### — ANNOTATIONS

|                                                                                                                                     |                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>ax.annotate("Label", xy=(x, y))</code>                                                                                        | Annotation sans flèche                                       |
| <code>ax.annotate("Label", xy=(x,y),<br/>xytext=(xt,yt),<br/>arrowprops=dict(arrowstyle="-&gt;"))</code>                            | Annotation avec flèche                                       |
| <code>ax.annotate("Label", xy=(x,y), xytext=(xt,yt),<br/>arrowprops=dict(arrowstyle="-&gt;", color="red"))</code>                   |                                                              |
| <code>ax.annotate("Label", xy=(x,y), xytext=(xt,yt),<br/>arrowprops=dict(arrowstyle="-&gt;", lw=2))</code>                          |                                                              |
| <code>ax.annotate("Label", xy=(x,y), xytext=(xt,yt),<br/>arrowprops=dict(arrowstyle="fancy",<br/>facecolor="black"))</code>         |                                                              |
| <code>ax.annotate("Label", xy=(x,y),<br/>xycoords="data",<br/>xytext=(0.5, 0.95),<br/>textcoords="axes fraction")</code>            | Coord. depuis les données (défaut)<br>Coord. texte relatives |
| <code>ax.annotate("Label", xy=(x,y), xytext=(xt,yt),<br/>fontsize=12, ha="center",<br/>bbox=dict(boxstyle="round4", fc="w"))</code> |                                                              |
| <code>ax.annotate("", xy=(x2,y2), xytext=(x1,y1),<br/>arrowprops=dict(arrowstyle="&lt;-&gt;"))</code>                               | Flèche sans texte                                            |
| # Styles de flèche : "<->", "<->", "<->", "<->", "<->", "<->", "<->", "<->", "<->", "<->"                                           |                                                              |

#### — LIGNES DE RÉFÉRENCE

|                                                                                       |                             |
|---------------------------------------------------------------------------------------|-----------------------------|
| <code>ax.axhline(y=0)</code>                                                          | Ligne horizontale à y=0     |
| <code>ax.axhline(y=0, color="r", linestyle="--")</code>                               | Avec style                  |
| <code>ax.axhline(y=0, xmin=0.2, xmax=0.8)</code>                                      | Partielle (fractions 0-1)   |
| <code>ax.axhline(y=0, linewidth=2, alpha=0.7)</code>                                  | Épaisseur et transparence   |
| <code>ax.axvline(x=0)</code>                                                          | Ligne verticale à x=0       |
| <code>ax.axvline(x=5, color="g", linestyle=":")</code>                                | Avec style                  |
| <code>ax.axvline(x=5, ymin=0.1, ymax=0.9)</code>                                      | Partielle                   |
| <code>ax.axhspan(ymin=0, ymax=1)</code>                                               | Zone horizontale colorée    |
| <code>ax.axhspan(ymin=0, ymax=1, alpha=0.2,<br/>facecolor="green")</code>             | Zone avec style             |
| <code>ax.axvspan(xmin=0, xmax=1)</code>                                               | Zone verticale colorée      |
| <code>ax.axvspan(xmin=2, xmax=4, alpha=0.2,<br/>facecolor="red", label="Zone")</code> | Zone avec légende           |
| <code>ax.axline((x1,y1), (x2,y2))</code>                                              | Droite passant par 2 points |
| <code>ax.axline((x,y), slope=m)</code>                                                | Droite par point et pente   |
| <code>ax.axline((x,y), slope=m, color="r")</code>                                     | Avec style                  |

#### — FORMES GÉOMÉTRIQUES

```

from matplotlib.patches import Rectangle, Circle, Ellipse, Polygon, FancyArrow
rect = Rectangle((x,y), width, height, fc="blue", ec="black", alpha=0.3)
ax.add_patch(rect)
circ = Circle((cx,cy), radius=r, fc="red", ec="black")
ax.add_patch(circ)
ell = Ellipse((cx,cy), width=w, height=h, angle=45, fc="green")
ax.add_patch(ell)
poly = Polygon([[x1,y1],[x2,y2],[x3,y3]], fc="yellow", ec="black")
ax.add_patch(poly)
arrow = FancyArrow(x, y, dx, dy, width=0.05, fc="purple")
ax.add_patch(arrow)

```

#### EXEMPLES :

```

ax.axhline(y=df["montant"].mean(), color="r", linestyle="--",
label=f"Moyenne = {df['montant'].mean():.1f}")
ax.annotate("Pic max", xy=(x_max, y_max), xytext=(x_max+1, y_max*0.9),
arrowprops=dict(arrowstyle="->", color="black"),

```

```

        fontsize=11, ha="center")
ax.text(0.02, 0.98, "n = 1234", transform=ax.transAxes,
        va="top", ha="left", fontsize=10)

```

## [plt/ax] Couleurs, colormaps et styles visuels

**DESCRIPTION** : Gestion des couleurs, palettes et styles visuels  
**POURQUOI** : Rendre les graphiques clairs, esthétiques et accessibles.  
**CONTEXTE** : Cohérence visuelle, accessibilité, publications.

### — SPÉCIFICATION DES COULEURS —

|                                       |                                  |
|---------------------------------------|----------------------------------|
| "blue"                                | Couleur nommée CSS               |
| "steelblue"                           | Couleur nommée avancée           |
| "#2ecc71"                             | Hexadécimal RGB                  |
| "#2ecc7180"                           | Hexadécimal RGBA                 |
| (0.2, 0.8, 0.4)                       | Tuple RGB (0-1)                  |
| (0.2, 0.8, 0.4, 0.5)                  | Tuple RGBA (0-1)                 |
| "C0", "C1", ..., "C9"                 | Couleurs du cycle courant        |
| "tab:blue", "tab:orange", "tab:green" | Palette Tableau                  |
| "0.5"                                 | Niveau de gris (0=noir, 1=blanc) |
| mcolors.to_rgba("steelblue")          | Convertit en RGBA                |
| mcolors.to_hex("steelblue")           | Convertit en hex                 |
| mcolors.CSS4_COLORS                   | Dict de toutes les couleurs CSS4 |
| mcolors.TABLEAU_COLORS                | Dict des couleurs Tableau        |

### — CYCLES DE COULEURS —

|                                                                 |                     |
|-----------------------------------------------------------------|---------------------|
| mpl.rcParams["axes.prop_cycle"]                                 | Cycle actuel        |
| from cycler import cycler                                       |                     |
| mpl.rcParams["axes.prop_cycle"] = cycler(color=["r", "g", "b"]) |                     |
| ax.set_prop_cycle(color=["r", "g", "b"])                        | Cycle pour cet axes |

### — COLORMAPS —

|                                                              |                                          |
|--------------------------------------------------------------|------------------------------------------|
| # Séquentielles                                              |                                          |
| "viridis"                                                    | Bleu → jaune (perceptuellement uniforme) |
| "plasma"                                                     | Violet → jaune                           |
| "magma"                                                      | Noir → blanc                             |
| "inferno"                                                    | Noir → jaune                             |
| "cividis"                                                    | Accessible daltoniens                    |
| "Blues", "Greens", "Reds", "Greys"                           | Monochromes                              |
| "YlOrRd", "YlGnBu", "PuBuGn"                                 | Multi-couleurs séquentielles             |
| # Divergentes                                                |                                          |
| "RdBu", "RdYlGn", "PiYG", "BrBG"                             | Divergentes symétriques                  |
| "coolwarm", "seismic", "bwr"                                 | Bleu-blanc-rouge                         |
| # Cycliques                                                  |                                          |
| "hsv", "twilight", "twilight_shifted"                        | Cycliques                                |
| # Qualitatives                                               |                                          |
| "Set1", "Set2", "Set3", "tab10", "tab20"                     | Qualitatives                             |
| "Pastell", "Pastel2", "Paired", "Accent"                     | Pastels                                  |
| # Inverser une colormap                                      |                                          |
| "viridis_r"                                                  | Version inversée de viridis              |
| cm.get_cmap("viridis")                                       | Objet colormap                           |
| cm.get_cmap("viridis", 10)                                   | Avec N couleurs discrètes                |
| cm.viridis(0.5)                                              | Couleur RGBA au niveau 0.5               |
| mpl.colormaps["viridis"]                                     | Accès dict aux colormaps                 |
| plt.colormaps()                                              | Liste toutes les colormaps               |
| # Normalisation pour colormaps                               |                                          |
| norm = mpl.colors.Normalize(vmin=0, vmax=100)                | Normalisation linéaire                   |
| norm = mpl.colors.LogNorm(vmin=1, vmax=1000)                 | Normalisation log                        |
| norm = mpl.colors.TwoSlopeNorm(vcenter=0, vmin=-10, vmax=20) | Normalisation bilatérale                 |
| norm = mpl.colors.BoundaryNorm([0,1,5,10], 4)                | Normalisation par seuils                 |

### — COLORBAR —

|                                                   |                                 |
|---------------------------------------------------|---------------------------------|
| plt.colorbar(im)                                  | Colorbar depuis un mappable     |
| plt.colorbar(im, ax=ax)                           | Attachée à un axes              |
| plt.colorbar(im, ax=ax, label="Valeur")           | Avec étiquette                  |
| plt.colorbar(im, ax=ax, orientation="horizontal") | Horizontale                     |
| plt.colorbar(im, ax=ax, shrink=0.8)               | Taille réduite (80%)            |
| plt.colorbar(im, ax=ax, aspect=20)                | Rapport d'aspect                |
| plt.colorbar(im, ax=ax, pad=0.05)                 | Espace axes ↔ colorbar          |
| plt.colorbar(im, ax=ax, fraction=0.046)           | Fraction de la taille de l'axes |
| plt.colorbar(im, ax=ax, ticks=[0,50,100])         | Ticks personnalisés             |
| plt.colorbar(im, ax=ax, format="%.1f")            | Format des étiquettes           |
| plt.colorbar(im, ax=ax, extend="max")             | Flèche au-delà du max           |

|                                                         |                                 |
|---------------------------------------------------------|---------------------------------|
| <code>plt.colorbar(im, ax=ax, extend="both")</code>     | Flèches au-delà des deux bornes |
| <code>cbar = plt.colorbar(im, ax=ax)</code>             | Objet colorbar                  |
| <code>cbar.set_label("Température", fontsize=12)</code> | Étiquette a posteriori          |
| <code>cbar.set_ticks([0,50,100])</code>                 | Ticks a posteriori              |
| <code>cbar.ax.tick_params(labelsize=10)</code>          | Taille des ticks                |

```

EXEMPLES      :
sc = ax.scatter(x, y, c=z, cmap="RdBu", vmin=-3, vmax=3, s=50)
cbar = plt.colorbar(sc, ax=ax, label="Valeur Z")
cbar.set_ticks([-3, -1, 0, 1, 3])

```

## [plt] Sauvegarde – savefig

**DESCRIPTION** : Sauvegarde la figure dans différents formats  
**POURQUOI** : Exporter les graphiques pour rapports, publications, présentations.  
**CONTEXTE** : Finalisation d'un graphique, automatisation de rapports.

### FONCTIONS PRINCIPALES :

|                                                                          |                                      |
|--------------------------------------------------------------------------|--------------------------------------|
| <code>plt.savefig("figure.png")</code>                                   | Sauvegarde en PNG                    |
| <code>plt.savefig("figure.pdf")</code>                                   | Sauvegarde en PDF (vectoriel)        |
| <code>plt.savefig("figure.svg")</code>                                   | Sauvegarde en SVG (vectoriel)        |
| <code>plt.savefig("figure.eps")</code>                                   | Sauvegarde en EPS (vectoriel)        |
| <code>plt.savefig("figure.jpg")</code>                                   | Sauvegarde en JPEG                   |
| <code>plt.savefig("figure.tiff")</code>                                  | Sauvegarde en TIFF                   |
| <code>plt.savefig("figure.webp")</code>                                  | Sauvegarde en WebP                   |
| <code>plt.savefig("fig.png", dpi=150)</code>                             | Résolution 150 DPI                   |
| <code>plt.savefig("fig.png", dpi=300)</code>                             | Haute résolution (publications)      |
| <code>plt.savefig("fig.png", dpi=72)</code>                              | Résolution web                       |
| <code>plt.savefig("fig.png", bbox_inches="tight")</code>                 | Recadrage auto (supprime les marges) |
| <code>plt.savefig("fig.png", bbox_inches="tight", pad_inches=0.1)</code> | Avec marge                           |
| <code>plt.savefig("fig.png", facecolor="white")</code>                   | Fond blanc explicite                 |
| <code>plt.savefig("fig.png", facecolor="none")</code>                    | Fond transparent                     |
| <code>plt.savefig("fig.png", edgecolor="none")</code>                    | Sans bord                            |
| <code>plt.savefig("fig.png", transparent=True)</code>                    | Arrière-plan transparent (PNG)       |
| <code>plt.savefig("fig.pdf", bbox_inches="tight")</code>                 | PDF sans marges excessives           |
| <code>plt.savefig("fig.pdf", metadata={"Author": "moi"})</code>          | Métadonnées PDF                      |
| <code>plt.savefig(buffer, format="png")</code>                           | Sauvegarde dans un buffer            |
| <code>plt.savefig(buffer, format="svg")</code>                           | Buffer SVG                           |
| <code>fig.savefig("fig.png", dpi=150)</code>                             | Méthode de la Figure                 |
| # Sauvegarde dans un buffer mémoire (utile pour web)                     |                                      |
| import io                                                                |                                      |
| buf = io.BytesIO()                                                       |                                      |
| plt.savefig(buf, format="png", bbox_inches="tight")                      |                                      |
| buf.seek(0)                                                              |                                      |

### FORMATS ET USAGES :

|      |                                                                |
|------|----------------------------------------------------------------|
| PNG  | → Web, présentations (raster, fond transparent possible)       |
| PDF  | → Publications scientifiques (vectoriel, texte sélectionnable) |
| SVG  | → Web interactif, édition Inkscape (vectoriel)                 |
| EPS  | → LaTeX, anciens systèmes d'impression (vectoriel)             |
| JPEG | → Photos, lorsque la taille du fichier est prioritaire         |
| TIFF | → Impression haute qualité, archivage                          |
| WebP | → Web moderne, bonne compression                               |

```

EXEMPLES      :
plt.tight_layout()
plt.savefig("rapport_mensuel.pdf", dpi=300, bbox_inches="tight", facecolor="white")
plt.savefig("aperçu_web.png", dpi=100, bbox_inches="tight", transparent=False)

```

## [ax] Axes secondaires et axes jumeaux – twinx, twiny, inset\_axes

**DESCRIPTION** : Crée des axes supplémentaires partageant une échelle  
**POURQUOI** : Superposer deux séries avec des unités ou des échelles différentes.  
**CONTEXTE** : Graphiques météo, finance, comparaisons multi-variables.

### FONCTIONS PRINCIPALES :

|                                            |                          |
|--------------------------------------------|--------------------------|
| <code>ax2 = ax.twinx()</code>              | Double axe Y (partage X) |
| <code>ax2 = ax.twiny()</code>              | Double axe X (partage Y) |
| <code>ax.twinx()</code>                    | Retourne le nouvel axes  |
| # Axes superposés                          |                          |
| <code>ax1.plot(x, y1, color="blue")</code> |                          |
| <code>ax2 = ax1.twinx()</code>             |                          |
| <code>ax2.plot(x, y2, color="red")</code>  |                          |

```

ax2.set_ylabel("Axe Y secondaire", color="red")
ax2.tick_params(axis="y", labelcolor="red")
# Axes insérés
from mpl_toolkits.axes_grid1.inset_locator import inset_axes
axins = inset_axes(ax, width="40%", height="30%", loc="upper right")
axins = inset_axes(ax, width=2.0, height=1.5, loc="lower left")
axins = ax.inset_axes([0.6, 0.1, 0.35, 0.35]) Via coordonnées relatives
axins.plot(x_zoom, y_zoom)
ax.indicate_inset_zoom(axins, edgecolor="black") Lien avec rectangle de zoom
# Axe secondaire (méthode moderne)
secax = ax.secondary_xaxis("top")           Axe X secondaire en haut
secax = ax.secondary_xaxis("bottom")        Axe X secondaire en bas
secax = ax.secondary_yaxis("right")         Axe Y secondaire à droite
secax = ax.secondary_yaxis("left")          Axe Y secondaire à gauche
secax = ax.secondary_xaxis("top",           Avec transformation
                          functions=(f_forward, f_inverse))
secax.set_xlabel("Degrés Celsius")

```

EXEMPLES :

```

fig, ax1 = plt.subplots()
ax1.bar(x, volume, color="steelblue", alpha=0.7, label="Volume")
ax2 = ax1.twinx()
ax2.plot(x, prix, color="red", lw=2, label="Prix")
ax1.set_ylabel("Volume", color="steelblue")
ax2.set_ylabel("Prix (€)", color="red")

```

[ax] Graphiques 3D – mpl\_toolkits.mplot3d

DESCRIPTION : Graphiques en trois dimensions  
 POURQUOI : Visualiser des données à trois variables ou des surfaces.  
 CONTEXTE : Mathématiques, sciences, optimisation, exploration de données.

SETUP :

```

from mpl_toolkits.mplot3d import Axes3D
fig = plt.figure()
ax = fig.add_subplot(projection="3d")      Axes 3D
ax = plt.axes(projection="3d")            Via pyplot
fig, ax = plt.subplots(subplot_kw={"projection": "3d"})

```

FONCTIONS PRINCIPALES :

```

ax.plot3D(x, y, z)           Courbe 3D
ax.plot3D(x, y, z, "r-o")    Avec style
ax.scatter3D(x, y, z)         Nuage de points 3D
ax.scatter3D(x, y, z, c=couleurs, cmap="viridis") Couleur variable
ax.scatter3D(x, y, z, s=tailles) Taille variable
ax.bar3d(x, y, z, dx, dy, dz) Barres 3D
ax.bar3d(x, y, z, dx, dy, dz, color="blue") Avec couleur
ax.plot_surface(X, Y, Z)      Surface 3D
ax.plot_surface(X, Y, Z, cmap="viridis") Avec colormap
ax.plot_surface(X, Y, Z, alpha=0.7) Avec transparence
ax.plot_surface(X, Y, Z, rstride=1, cstride=1) Résolution de la surface
ax.plot_wireframe(X, Y, Z)    Surface en fil de fer
ax.plot_wireframe(X, Y, Z, color="black", linewidth=0.5)
ax.contour3D(X, Y, Z, 50)     Contours 3D (50 niveaux)
ax.contourf(X, Y, Z, zdir="z", offset=-1) Contour projeté sur plan Z
ax.quiver3D(X, Y, Z, U, V, W) Champ de vecteurs 3D
ax.set_xlabel("X"), ax.set_ylabel("Y"), ax.set_zlabel("Z")
ax.set_xlim(0,10), ax.set_ylim(0,10), ax.set_zlim(0,10)
ax.set_title("Surface 3D")
ax.view_init(elev=30, azimuth=45) Angle de vue (élévation, azimuth)
ax.view_init(elev=90, azimuth=0)  Vue de dessus
ax.dist = 10                       Distance de la caméra
ax.set_box_aspect([1,1,0.5])      Rapport d'aspect 3D
ax.grid(True)                      Grille 3D

```

EXEMPLES :

```

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(projection="3d")
X, Y = np.meshgrid(np.linspace(-5,5,50), np.linspace(-5,5,50))
Z = np.sin(np.sqrt(X**2 + Y**2))
surf = ax.plot_surface(X, Y, Z, cmap="viridis", alpha=0.8)
fig.colorbar(surf, ax=ax, shrink=0.5)
ax.view_init(elev=25, azimuth=135)

```

[ax] Graphiques polaires – projection="polar"

DESCRIPTION : Graphiques en coordonnées polaires  
POURQUOI : Données angulaires, roses des vents, diagrammes radar.  
CONTEXTE : Météorologie, navigation, analyse de données circulaires.

SETUP :  
ax = plt.subplot(projection="polar")  
fig, ax = plt.subplots(subplot\_kw={"projection": "polar"})

FONCTIONS PRINCIPALES :

|                                                                                     |                                     |
|-------------------------------------------------------------------------------------|-------------------------------------|
| ax.plot(theta, r)                                                                   | Courbe polaire                      |
| ax.scatter(theta, r)                                                                | Points en polaire                   |
| ax.bar(theta, r)                                                                    | Barres polaires (rose des vents)    |
| ax.bar(theta, r, width=np.pi/6)                                                     | Largeur angulaire des barres        |
| ax.fill(theta, r, alpha=0.3)                                                        | Zone remplie en polaire             |
| ax.set_theta_zero_location("N")                                                     | 0° en haut (Nord)                   |
| ax.set_theta_zero_location("E")                                                     | 0° à droite (Est, défaut)           |
| ax.set_theta_direction(1)                                                           | Sens antihoraire (défaut)           |
| ax.set_theta_direction(-1)                                                          | Sens horaire                        |
| ax.set_thetagrids(range(0,360,30))                                                  | Graduations angulaires tous les 30° |
| ax.set_thetagrids(range(0,360,45),<br>labels=["N","NE","E","SE","S","SO","O","NO"]) | Avec étiquettes personnalisées      |
| ax.set_rgrids([0.2, 0.4, 0.6, 0.8, 1.0])                                            | Graduations radiales                |
| ax.set_rmax(1.0)                                                                    | Rayon maximum                       |
| ax.set_rmin(0)                                                                      | Rayon minimum                       |
| ax.set_rlabel_position(90)                                                          | Position des étiquettes radiales    |
| ax.set_rticks([0.5, 1.0])                                                           | Ticks radiaux                       |
| ax.set_rlim(0, 1.5)                                                                 | Limites radiales                    |

[plt] Animations – FuncAnimation, ArtistAnimation

DESCRIPTION : Crée des animations à partir de figures matplotlib  
POURQUOI : Visualiser des évolutions temporelles ou des processus dynamiques.  
CONTEXTE : Simulations, évolutions temporelles, tutoriels visuels.

IMPORTS :  
from matplotlib.animation import FuncAnimation, ArtistAnimation

FONCTIONS PRINCIPALES :

```
# FuncAnimation – fonction de mise à jour appelée à chaque frame
ani = FuncAnimation(fig, update_func, frames=100)
ani = FuncAnimation(fig, update_func, frames=range(0,100))
ani = FuncAnimation(fig, update_func, frames=iterable)
ani = FuncAnimation(fig, update_func, frames=100, interval=50)  ms entre frames
ani = FuncAnimation(fig, update_func, frames=100, blit=True)    Optimisé
ani = FuncAnimation(fig, update_func, frames=100, repeat=True) Boucle
ani = FuncAnimation(fig, update_func, frames=100, repeat_delay=1000) Pause ms
ani = FuncAnimation(fig, update_func, init_func=init)           Fonction init
ani = FuncAnimation(fig, update_func, fargs=(arg1, arg2))       Arguments
# ArtistAnimation – liste de frames pré-calculées
frames = []
for i in range(10):
    line, = ax.plot(x, y[i])
    frames.append([line])
ani = ArtistAnimation(fig, frames, interval=100)
# Sauvegarde de l'animation
ani.save("animation.gif", writer="pillow", fps=30)
ani.save("animation.gif", writer="pillow", dpi=100)
ani.save("animation.mp4", writer="ffmpeg", fps=30)
ani.save("animation.mp4", writer="ffmpeg", bitrate=1800)
ani.to_html5_video()      Exporte en HTML5 (Jupyter)
ani.to_jshtml()           Exporte en JS interactif
plt.rcParams["animation.embed_limit"] = 50  Limite taille (MB) pour embed
```

[plt/ax] Tableaux dans les graphiques – table

DESCRIPTION : Insère un tableau dans une figure matplotlib  
POURQUOI : Afficher des données chiffrées à côté du graphique.  
CONTEXTE : Rapports, dashboards, synthèses.

FONCTIONS PRINCIPALES :

```
tab = ax.table(cellText=données, colLabels=colonnes, loc="bottom")
tab = ax.table(cellText=données, colLabels=colonnes, loc="top")
```

```

tab = ax.table(cellText=données, colLabels=colonnes, loc="center")
tab = ax.table(cellText=données, rowLabels=lignes, loc="right")
tab = ax.table(cellText=données,
               colLabels=colonnes,
               cellColours=couleurs,      Couleurs des cellules
               loc="bottom")
tab.auto_set_font_size(False)             Désactive auto-taille
tab.set_fontsize(10)                     Taille de police
tab.auto_set_column_width([0,1,2,3])      Ajuste les largeurs de colonnes
tab.scale(1, 1.5)                         Redimensionne (x, y)
tab[(0, 0)].set_facecolor("steelblue")    Couleur d'une cellule
tab[(0, 0)].set_text_props(color="white")  Couleur du texte d'une cellule
plt.table(cellText=données, ...)          Via pyplot

```

[plt/ax] Layouts et composition – tight\_layout, constrained\_layout

DESCRIPTION : Contrôle la mise en page et l'espace des éléments  
 POURQUOI : Éviter les superpositions de titres, étiquettes et graphiques.  
 CONTEXTE : Graphiques multi-panneaux, publications, présentations.

#### FONCTIONS PRINCIPALES :

```

plt.tight_layout()           Ajuste auto les marges
plt.tight_layout(pad=1.0)    Rembourrage global
plt.tight_layout(h_pad=2.0)  Rembourrage vertical entre axes
plt.tight_layout(w_pad=2.0)  Rembourrage horizontal entre axes
plt.tight_layout(rect=[0,0,1,0.95]) Région du layout (pour supitle)
plt.subplots_adjust(         Contrôle précis des marges
    left=0.1,                Marge gauche
    right=0.95,              Marge droite
    bottom=0.1,              Marge basse
    top=0.95,                Marge haute
    hspace=0.4,              Espace vertical entre sous-graphiques
    wspace=0.3)              Espace horizontal
fig.set_constrained_layout(True) Layout contraint activé
fig.set_constrained_layout_pads( Paramètres du layout contraint
    hspace=0.1, wspace=0.1,
    h_pad=0.05, w_pad=0.05)
fig.suptitle("Titre global", y=0.98) Titre de la figure
fig.suptitle("Titre", fontsize=16,
             fontweight="bold")
# Espacement entre un supitle et les sous-graphiques
fig.suptitle("Titre")
fig.tight_layout(rect=[0, 0, 1, 0.96]) Laisse de la place pour supitle

```

#### RÉCAPITULATIF DES TYPES DE GRAPHIQUES

| TYPE DE DONNÉES          | GRAPHIQUE RECOMMANDÉ | FONCTION                   |
|--------------------------|----------------------|----------------------------|
| Évolution temporelle     | Courbe               | ax.plot()                  |
| Relation X/Y             | Nuage de points      | ax.scatter()               |
| Comparaison catégories   | Barres verticales    | ax.bar()                   |
| Comparaison catégories   | Barres horizontales  | ax.barh()                  |
| Distribution continue    | Histogramme          | ax.hist()                  |
| Distribution + quartiles | Boîte à moustaches   | ax.boxplot()               |
| Distribution (forme)     | Violon               | ax.violinplot()            |
| Composition (%)          | Camembert            | ax.pie()                   |
| Composition (empilé)     | Barres empilées      | ax.bar(..., bottom=)       |
| Incertitude              | Zone remplie         | ax.fill_between()          |
| Mesures + erreurs        | Barres d'erreur      | ax.errorbar()              |
| Deux échelles            | Axes jumeaux         | ax.twinx()                 |
| Matrice de valeurs       | Heatmap / imshow     | ax.imshow() / pcolormesh() |
| Données géographiques    | Contour              | ax.contourf()              |
| Densité 2D               | Hexbin               | ax.hexbin()                |
| Données circulaires      | Polaire              | projection="polar"         |
| 3 variables              | Surface / 3D         | projection="3d"            |
| Données discrètes        | Tiges / bâtons       | ax.stem()                  |
| Événements               | Eventplot            | ax.eventplot()             |
| Vecteurs                 | Quiver               | ax.quiver()                |

— RÉCAPITULATIF DES POSITIONS DE LÉGENDE —  
 CODE POSITION  
 "best" Meilleure position automatique

|                |                                                |
|----------------|------------------------------------------------|
| "upper right"  | En haut à droite (défaut si best non spécifié) |
| "upper left"   | En haut à gauche                               |
| "lower left"   | En bas à gauche                                |
| "lower right"  | En bas à droite                                |
| "right"        | À droite (milieu)                              |
| "center left"  | À gauche (milieu)                              |
| "center right" | À droite (milieu)                              |
| "lower center" | En bas (milieu)                                |
| "upper center" | En haut (milieu)                               |
| "center"       | Au centre                                      |
| (x, y)         | Position manuelle (0-1, 0-1)                   |
| bbox_to_anchor | Hors du graphique                              |

---

#### — RÉCAPITULATIF DES FORMATS DE LIGNE

| CODE   | SIGNIFICATION  |
|--------|----------------|
| "-"    | Ligne continue |
| "--"   | Tirets         |
| "-."   | Tiret-point    |
| ":"    | Pointillés     |
| "None" | Aucune ligne   |

---

#### — RÉCAPITULATIF DES MARQUEURS

| CODE | FORME             |
|------|-------------------|
| "."  | Point             |
| ","  | Pixel             |
| "o"  | Cercle            |
| "v"  | Triangle bas      |
| "^"  | Triangle haut     |
| "<"  | Triangle gauche   |
| ">"  | Triangle droit    |
| "s"  | Carré             |
| "p"  | Pentagone         |
| "*"  | Étoile            |
| "h"  | Hexagone          |
| "+"  | Plus              |
| "x"  | Croix             |
| "D"  | Diamant           |
| "d"  | Diamant fin       |
| " "  | Ligne verticale   |
| "_"  | Ligne horizontale |

---

#### — NOTATION COMPACTE FORMAT STRING

|       |                          |
|-------|--------------------------|
| "b-"  | Bleu, ligne continue     |
| "r--" | Rouge, tirets            |
| "g-o" | Vert, ligne avec cercles |
| "k:"  | Noir, pointillés         |
| "ms"  | Magenta, carré           |

FORMAT = [couleur][marqueur][ligne] (tous optionnels)  
Couleurs courtes : b=bleu, g=vert, r=rouge, c=cyan, m=magenta, y=jaune, k=noir, w=blanc

=====